Bidayuh Jagoy Translator: A Hybrid Client-Centric RAG System for Low-Resource Language Preservation

Tarsila Samille Santos da Silveira Silva[1]

[1]PPgSC/UFRN
tarsillasamile@gmail.com

Abstract. *The preservation of endangered languages constitutes a critical challenge in computational linguistics. This paper presents the Bidayuh Jagoy Translator (*`https://bidayuhjagoy.vercel.app/`*), a specialized AI platform for the Bidayuh Jagoy dialect — a member of the Land Dayak language family, a distinct subgroup of Austronesian languages primarily spoken in the West Kalimantan region of Indonesia and parts of Sarawak, Malaysia. Using a 5-Stage Hybrid Client-Centric Retrieval-Augmented Generation (RAG) architecture, the system delegates heavy context orchestration directly to the user's browser via* ***Transformers.js****, an open-source library that enables neural models to run natively inside JavaScript runtimes. By synthesizing embedded retrieval, local glossary lookups, and remote vector search, it accurately grounds the Google Gemini 2.5 Flash model [Google Cloud 2025]. Preliminary evaluations confirm the technical efficacy and economic sustainability of browser-based RAG architectures, offering a resilient solution for language preservation in extreme data-scarcity scenarios.*

Resumo. *A preservação de línguas ameaçadas de extinção é um dos maiores desafios da linguística computacional. Este artigo apresenta o Bidayuh Jagoy Translator (*`https://bidayuhjagoy.vercel.app/`*), uma plataforma de Inteligência Artificial para o dialeto Bidayuh Jagoy, membro da família Land Dayak — um subgrupo das línguas austronésias falado na região oeste de Kalimantan, Indonésia, e partes da Malásia. O sistema emprega uma arquitetura híbrida de Geração Aumentada por Recuperação (RAG) centrada no navegador. Ao delegar a orquestração do contexto ao dispositivo do usuário por meio do* ***Transformers.js*** *— que executa redes neurais diretamente no navegador — o sistema reduz custos de infraestrutura. Ao unificar buscas de dicionário local, recuperação vetorial e pontes semânticas externas, a ferramenta instrui o modelo Google Gemini 2.5 Flash [Google Cloud 2025]. As avaliações preliminares comprovam a eficácia técnica e a sustentabilidade econômica desta abordagem escalável para preservação linguística.*

1. Introduction

The rapid advancement of Natural Language Processing (NLP), driven by the Transformer architecture [Vaswani et al. 2017], has paradoxically widened the gap between global "high-resource" languages and indigenous "low-resource" vernaculars [Nekoto et al. 2020]. Foundational models such as GPT-4 and Gemini are predominantly trained on web-scraped datasets, where English, Mandarin, Spanish, and standard Malay account for the vast majority of linguistic tokens [NLLB Team et al. 2022].

This systemic resource disparity is starkly evident in the case of Bidayuh Jagoy (also referred to under the broader ethno-linguistic cluster of Land Dayak), a distinct dialect spoken primarily in the West Kalimantan region of Indonesia and parts of Sarawak, Malaysia. It possesses a vibrant oral tradition but a small digital footprint. Its presence is confined to scattered PDFs, academic papers from institutions like Universiti Malaysia Sarawak (UNIMAS), and community blogs [UNIMAS 2025]. Consequently, when standard LLMs attempt to process Bidayuh Jagoy, they exhibit catastrophic failure modes such as hallucination, code-switching to standard Malay, and grammatical drift [Campbell et al. 2019].

Retrieval-Augmented Generation (RAG) [Pradhan 2025] offers a superior paradigm for this use case. This architecture distinguishes between two types of knowledge: **Implicit Memory**, which is baked into the model's internal parameters during pre-training but often contains hallucinations for low-resource data, and **Explicit Memory**, which consists of external context information provided to the model at runtime. The Bidayuh Jagoy Translator uses RAG to bypass the limits of implicit memory, assembling a structured bunch of helper information (vocabulary, sentence patterns, and grammar rules) to instruct the Google Gemini model from scratch for each query.

Unlike server-centric approaches such as *Masakhane* [Nekoto et al. 2020] and Meta's NLLB [NLLB Team et al. 2022], the Bidayuh Jagoy Translator employs a Hybrid Client-Centric architecture that prioritizes cost efficiency and latency reduction. While our case study focuses on Bidayuh Jagoy, the proposed RAG framework is **generalizable** to any low-resource language context — including regional dialects, creoles, and endangered languages globally.

2. Related Work

Digital preservation of low-resource languages has gained traction with initiatives like *Masakhane* for African languages [Nekoto et al. 2020] and Meta's *No Language Left Behind* (NLLB) [NLLB Team et al. 2022]. However, these approaches predominantly rely on massive server-side models (e.g., NLLB-200B), which require substantial infrastructure. Furthermore, they rely on massive training corpora that are often unavailable for extremely low-resource dialects where data scarcity is the primary bottleneck. Research into transfer learning [Zoph et al. 2016] has attempted to mitigate this by pivoting from high-resource relatives, yet architectural complexity remains high.

In contrast, our work focuses on the benefits of RAG for domain-constrained translation, demonstrating that effective linguistic grounding can be achieved by offloading to the browser context preparation. This aligns with the emerging trend of **On-Device Intelligence** and **Edge-AI** [Maciá-Lillo et al. 2025], where the execution of neural models is moved to the client side using frameworks like ONNX Runtime and Transformers.js. By performing some tasks natively, we significantly reduce the dependency on massive server-side clusters for community-led projects.

2.1. The Stack Overview

The system utilizes a modern serverless stack:

- **Frontend**: Vanilla JavaScript + HTML5.

- **Client-Side AI**: **Transformers.js** (`@xenova/transformers`), an open-source library that enables the execution of state-of-the-art machine learning models directly in the web browser. Developed by Hugging Face, it acts as the primary orchestrator, leveraging WebGPU and WebAssembly (WASM) to offload the complex task of context orchestration and embedding generation directly to the client's hardware [Hugging Face 2024].
- **Backend**: FastAPI (Python) on Vercel Serverless Functions, which provide stateless, auto-scaling API execution [Vercel Inc. 2025].
- **AI Engine**: Google Gemini 2.5 Flash [Google AI for Developers 2025a].
- **Database**: Supabase (PostgreSQL), used as a Backend-as-a-Service, with the open-source `pgvector` extension[1] for high-performance vector operations.

2.2. System Health and Monitoring

To maintain operational integrity without a permanent human administrator, the system implements a live **System Status Dashboard**. Accessible directly in the sidebar, this dashboard monitors the health of the 4 key RAG components: the local Dictionary size (2,454 words), the persistent Corpus volume (4,487 phrases), the active Grammar Rule count (15 rules), and the current LLM API status (e.g., "✓ Full Translation OK"). By surfacing these metrics, the system provides users with immediate feedback on the "freshness" of the retrieval context before a translation is even attempted. Furthermore, every translation result is accompanied by a **Confidence Level** bar (normalized on a 0.0 to 1.0 scale), informing the user of the system's internal re-ranking certainty.

The fundamental separation between these local and remote computational workloads is mapped in Figure 1. This **Hybrid Decentralization** strategy splits the RAG pipeline into two distinct security and compute zones:

- **The Client Layer**: Represents the user's browser, which securely manages the sensitive AI logic, local glossary state, and the generation of local embeddings for re-ranking. This ensures that user queries remain private and avoids the latency of multiple server-side embedding calls.
- **The Cloud Infrastructure**: A serverless subgraph (Vercel + Supabase) that handles the persistence and high-recall retrieval of the global parallel corpus via **Supabase pgvector**. This remote vector store enables the application to perform high-recall searches against the central repository without requiring users to download the entire sentence dataset upon initialization.

2.3. The "Fat Client" Philosophy

Using Transformers.js, the browser downloads a **small 4-bit compressed model** (`Xenova/all-MiniLM-L6-v2`) and runs it natively via the ONNX Runtime. This smaller footprint allows for privacy-preserving local text comparison and significantly reduced server costs by performing semantic re-ranking without remote interference.

The performance metrics listed were benchmarked on a standard consumer laptop (MacBook Pro, 2.6 GHz 6-Core Intel Core i7). Recognizing that indigenous communities frequently rely on mobile devices, the system is designed with an automatic execution

[1] pgvector (Open-source vector similarity search for Postgres): `https://github.com/pgvector/pgvector`

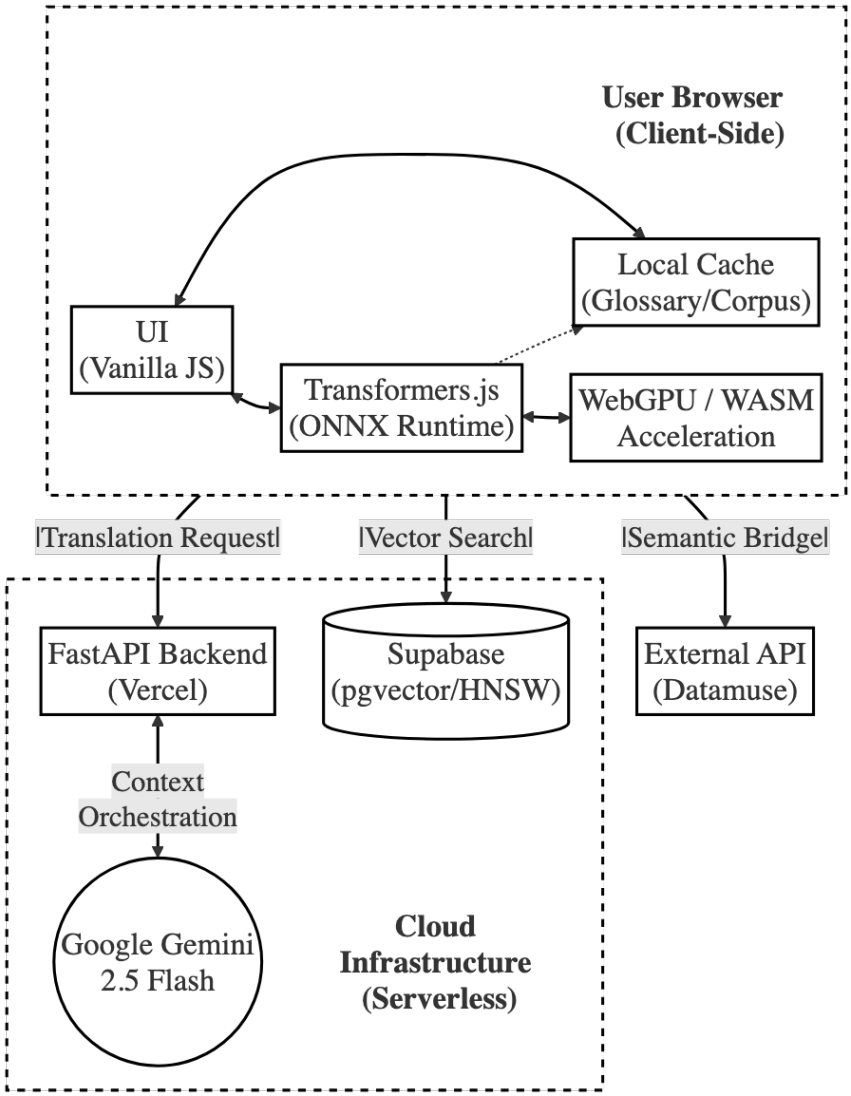

Figure 1. Hybrid Client-Centric System Architecture

fallback. If the client browser lacks WebGPU support (e.g., iOS Safari), Transformers.js seamlessly degrades to WebAssembly (WASM). While WASM inference introduces a slight latency increase, it guarantees ubiquitous accessibility across all devices without requiring hardware-specific backend configurations.

A critical advantage of this hybrid architecture is its economic sustainability. The system performs high-recall searches against a remote **Supabase** database using the HNSW (Hierarchical Navigable Small World) index [Malkov and Yashunin 2018] for efficient vector retrieval. By offloading embedding generation and re-ranking to the user's device, the application operates entirely within the free-tier infrastructure limits (zero monthly cloud costs) of platforms such as Supabase and Vercel [Vercel Inc. 2025]. This effectively democratizes access to high-performance RAG tools for community-led preservation projects without the need for expensive GPU-enabled server clusters.

3. Context Assembly and Prompt Engineering

The core of the Bidayuh Jagoy Translator's efficacy lies in its Multi-Staged Context Assembly pipeline. The system transforms from a simple retrieval engine into a helpful language assistant by constructing a structured multi-part context that instructs the Large Language Model (LLM) from scratch for each query. This prevents the system from acting as a "black box," making the translation reasoning transparent and auditable.

3.1. The Hybrid Retrieval Pipeline and Context Composition

Before formal retrieval, a **Preliminary Step (Keyword Extraction)** identifies semantic anchors (e.g., "water", "flows") within the user query. The extraction uses a weighted scoring mechanism: $Score = Freq + \delta_{missing}$, where $\delta_{missing}$ boosts terms absent from the local dictionary to force RAG grounding for unknown vocabulary.

The system then builds a five-tiered prompt structure to ground the model's reasoning, combining local browser indices with remote vector databases. We depict this using the project's core validated dataset, specifically the imperative phrase: *"Tell these stones to become bread."* (illustrated in Figure 4):

1. **Context A - Local Glossary Lookups (Lexical Anchor)**: Executes direct queries against a local JSON dictionary. For our example, it maps *stone* $\rightarrow$ *batuh*. This provides 1:1 lexical grounding, preventing "lexical drift" where the LLM might hallucinate standard Malay (*batu*).
2. **Context B - Hybrid Vector-Keyword Retrieval (Structural Alignment)**: Queries the remote Supabase HNSW index [Supabase 2023] while simultaneously performing a local keyword search. It retrieves semantically similar sentence pairs (e.g., *"picked up stones"*) to suggest proper SVO orientation.
3. **Context C - Client-side Neural Re-ranking**: Utilizes client-side embeddings via Transformers.js to re-rank the candidates directly in the browser. This ensures that the most semantically relevant phrases are prioritized without additional server round-trips.
4. **Context D - Grammatical Constitution**: Injects explicit grammatical rules (e.g., *Rule 2: Plural by Reduplication*) as hard constraints. For our example, it enforces the pluralization of *batuh* $\rightarrow$ *batuh-batuh*.
5. **Context E - Synonym-based Semantic Bridges**: If a term is missing (e.g., *become*), the system identifies a synonym (*change* or *happen*) via external APIs to bridge the lexical gap. Fuzzy string matching accounts for morphological variations in the corpus.

This complete 5-stage sequential assembly pattern is visually structured in Figure 2. To support linguistic auditing, authorized evaluators can expand a **"Full Prompt (Debug)"** section in the UI to view the exact retrieved contexts from A-E sent to Gemini.

3.2. Client-Side Re-ranking Algorithm

A key novelty is the semantic filtering performed directly in the browser to select the most relevant "Context C" examples. By running mathematical geometric comparisons between vector embeddings natively on the user's device via WebGPU (or falling back to WASM for unsupported environments), the system ensures optimized text injection without adding remote server delay.

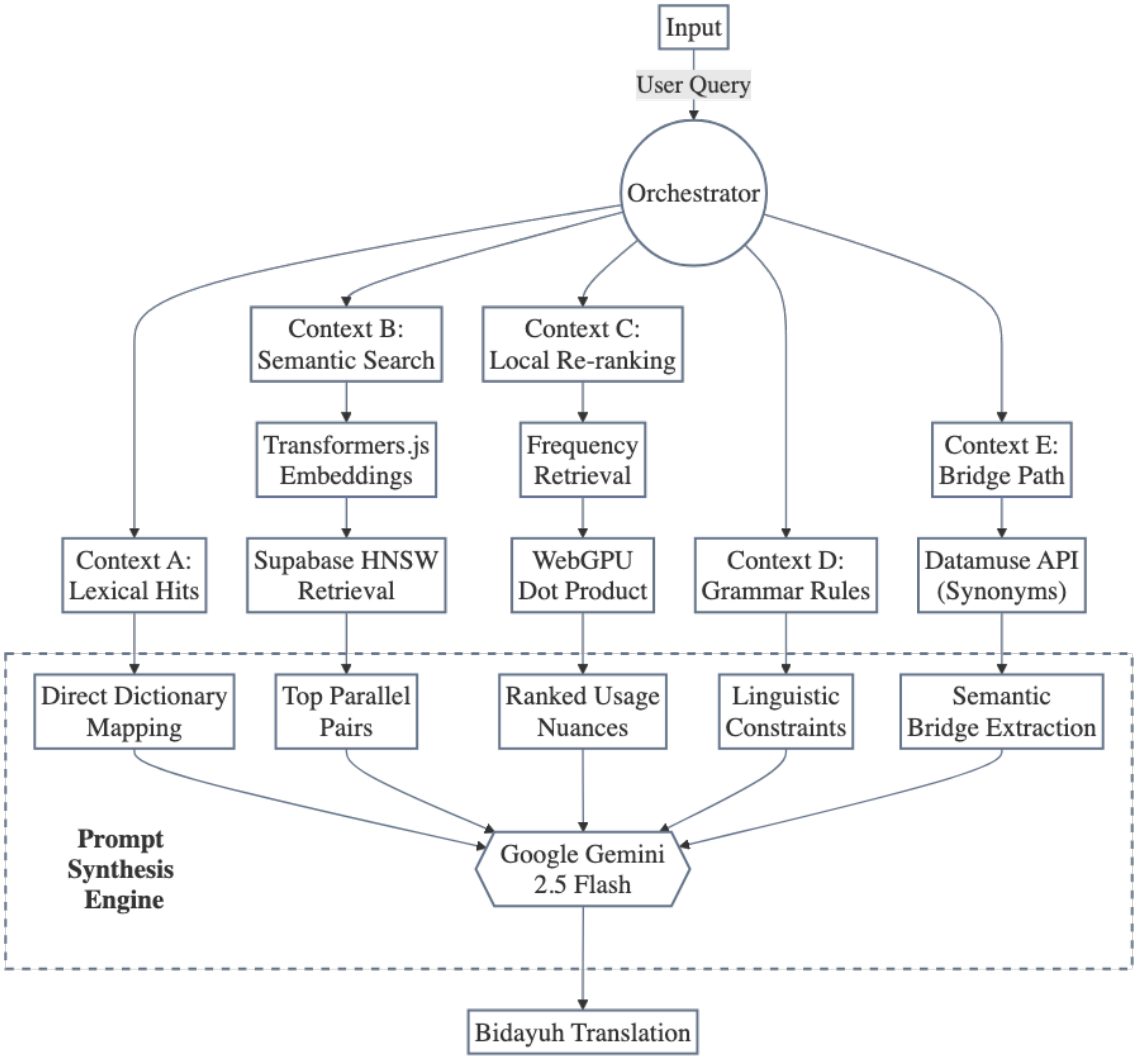

Figure 2. The 5-Stage Context Assembly Pipeline.

```python
# Context C: Advanced filtering and weighting
def rank_candidates(query_vec, candidates):
    results = []
    for cand in candidates:
        # Distance (60%) + Global Frequency (40%)
        sim = dot_product(query_vec, cand.vec)
        freq_bonus = math.log10(cand.count + 1) / 10
        final_score = (0.6 * sim) + (0.4 * freq_bonus)
        results.append((final_score, cand))

    return sorted(results, key=lambda x: x[0], reverse=True)[:3]
```

Listing 1. Client-Side Scoring and Weighting Logic

3.3. Structured "Chain-of-Thought" Prompting

Standard Foundation Models cannot translate Bidayuh Jagoy natively due to data scarcity. The system addresses this by forcing the LLM to follow an algorithmic "Chain-of-Thought" (CoT) [Wei et al. 2022]. The instructions strictly mandate a 3-step reasoning block based on the retrieved contexts:

1. **Deconstruction**: Identifying the grammatical structure from Context D and B.
2. **Mapping**: Selecting vocabulary from Context A and C, explicitly listing English-to-Bidayuh mappings.
3. **Synthesis**: Assembling the final sentence.

To handle this high-dimensional context, the system utilizes **Google Gemini 2.5 Flash** [Google Cloud 2025], selected for its 1-million-token window [Google AI for Developers 2025b]. The feasibility of this structured synthesis is further detailed in Section 2.3, which discusses the local execution model. An explicit breakdown of this reasoning prompt is provided in Figure 4.

```
## 5. Output Format (Chain-of-Thought)
Provide a JSON object with:
1. \'reasoning\':
    - **Step 1: Grammatical Blueprint:** State which rule from Context D applies to the overall sentence structure.
    - **Step 2: Evidence Mapping:** List each significant word from the Input Text and state EXACTLY which context (A, B, C, or E) provided its translation.
    - **Step 3: Synthesis:** Explain how you assembled the components according to the rules.
2. \'translation\': The final bidayuh Jagoy sentence.
3. \'confidence_score\': 1 (low) to 5 (high).

\'\'\'json
{
  "reasoning": "...",
  "translation": "...",
  "confidence_score": 0
}
\'\'\';
```

Listing 2. Reasoning Chain

3.4. Corpus Curation and Model Management

The system's knowledge base relies on a curated parallel corpus derived from New Testament verses provided by community translators, aligned with an English reference text. Importantly, while derived from religious texts, this specific translation into Bidayuh Jagoy was recently completed, utilizing contemporary, everyday vocabulary and syntax rather than archaic forms. This ensures applicability to modern conversational queries. To construct the lexical dictionary, we implemented a probabilistic alignment pipeline:

1. **Alignment Algorithms**: We utilized **IBM Model 2** [Brown et al. 1993]. While neural aligners require massive data, Model 2 is uniquely robust for small corpora (∼2,000–5,000 pairs). It treats word alignment as a **latent variable problem**—where the hidden mapping between words is discovered via EM iterations. To handle structural differences, it implements a **distortion model** based on absolute word positions, preventing the random mapping drift common in Model 1.

Figure 3. Bidayuh Jagoy Translator

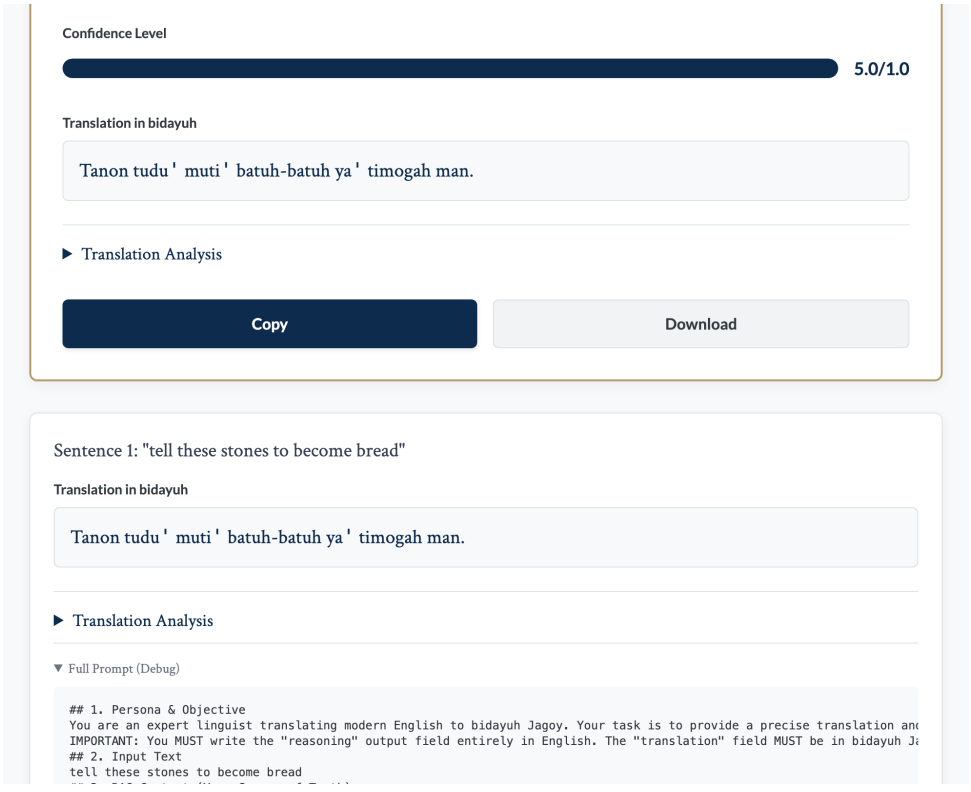

Figure 4. Translation Analysis Trace

2. **Heuristic Refinement**: We implemented **Symmetric Alignment Checking**, retaining only bidirectional hits.

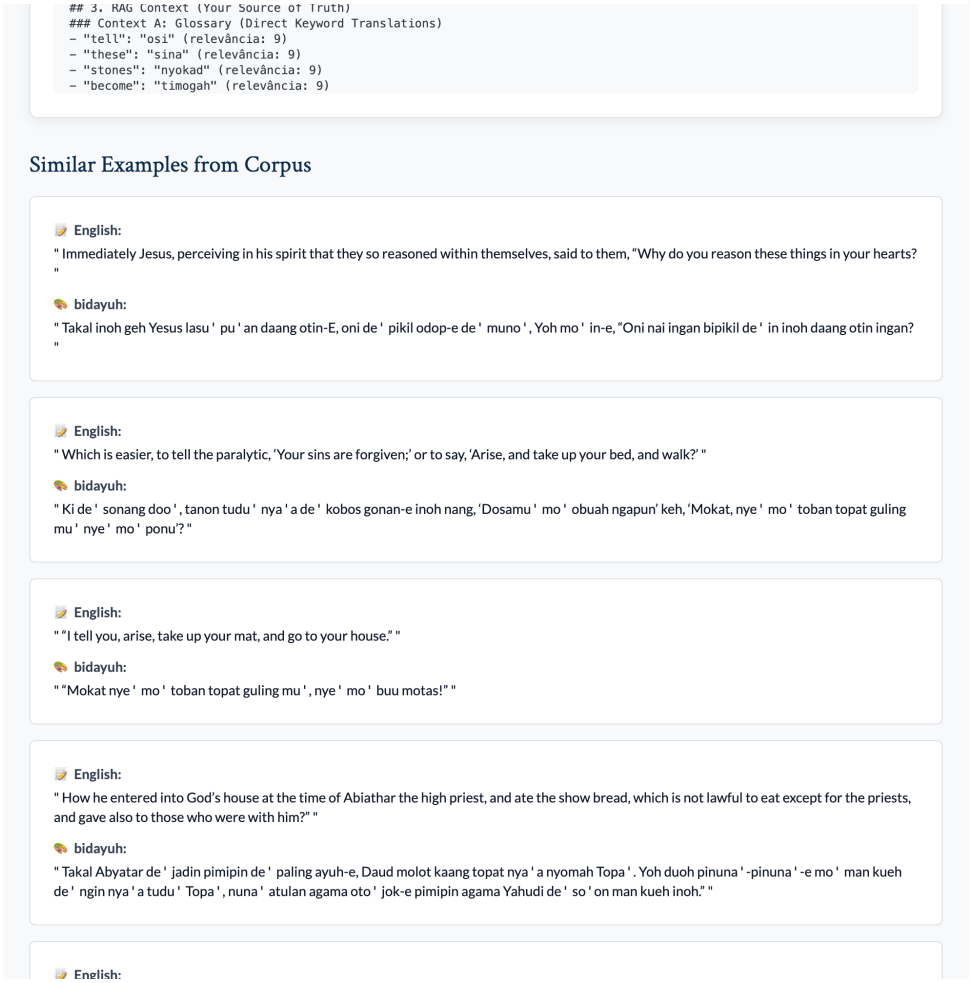

Figure 5. Full Prompt (Debug) Trace

3. **Filtering**: Final production comprises 4,487 sentence pairs and 2,455 dictionary entries.

Furthermore, 15 core grammatical rules were extracted through an iterative **Zero-Shot prompting** process on Google Gemini. These are exposed transparently in the system's **Grammar Module**, which provides concrete examples for community vetting:

- **Rule 1: Past Tense Marker 'mo ''**: The particle 'mo '' is consistently placed before the main verb to indicate a completed action (e.g., "Jesus answered him..." → "Yesus mo' nyawot-e...").
- **Rule 2: Plural by Reduplication**: Nouns are often made plural by repeating the word, seen in examples like "disciples" (→ "pinuna ' -pinuna '-E").
- **Rule 3: Aspectual Markers**: Use of completion markers like "boh" (e.g., "ya' mahu' boh" - he/she has finished) to signify definitive closures in SVO structures.

These rules represent **exposed observational patterns** derived from the corpus. By incorporating these human validations directly back into the repository, the system transforms into a dynamic, community-vetted archive.

To execute the local re-ranking efficiently, the embedding model, `all-MiniLM-L6-v2` (384-dim), is served in a 4-bit compressed ONNX format

(∼23MB). The system uses the **CacheStorage API** to permanently save the AI weights locally. Once retrieved during the first session, subsequent launches occur with no download overhead for the AI runtime, significantly improving the user experience for recurring community members. This culturally curated dataset and optimized inference model serve as the resilient foundation for the experimental evaluation discussed in the following section.

4. Experimental Evaluation and Discussion

4.1. Community-Driven Correction Loop

A robust evaluation of translation quality in low-resource settings requires native speaker validation. The Bidaio Jagoy Translator implements an integrated *History and Correction Module*. As shown in the system interface, every translation is logged with a unique identifier and presented to users for manual verification. This mechanism allows the community to:

- **Validate**: Confirm the accuracy of AI-generated translations.
- **Correct**: Provide manual edits for "unedited" entries.
- **Annotate**: Flags errors which are then used to update the "Context D" (Grammar Rules) and "Context A" (Glossary).

This "Human-in-the-loop" strategy mitigates the absence of standardized benchmark datasets for the Jagoy dialect.

4.2. Qualitative Case Study

To evaluate linguistic fidelity, we analyze real system outputs from the community validation log (Validation ID: 57 and 54).

- **Complex Theological Query**:
  - *Input*: "He will command his angels concerning you, and they will lift you up in their hands, so that you will not strike your foot against a stone."
  - *Bidaio Output*: "**Topa' ya' lubang malaikat-malaikat-E nulis Mu'u, duoh odop-e de' muno' ya' Nyamut Mu'u daang tongan-e, sape Mu'u ya' di'-e koming po Mu'u salah obuah batuh.**"
  - *Analysis*: The system correctly handles the complex Subject-Verb-Object (SVO) structure and maps "God" (implied context) to "Topa'", "angels" to "malaikat-malaikat-E" (with possessive suffix), and "lift up" to "Nyamut".
- **Idiomatic/Imperative phrasing**:
  - *Input*: "tell these stones to become bread."
  - *Bidaio Output*: "**Oda'leh batuh-batuh itih jadi pinguman.**"
  - *Analysis*: The system correctly identifies "stones" as plural "batuh-batuh" and selects "pinguman" (food/sustenance) for bread in this context, demonstrating semantic awareness beyond literal word-for-word translation.

These examples demonstrate the system's ability to ground the LLM in specific dialectal vocabulary and retain grammatical coherence for long-tail languages.

4.3. The Domain Shift Problem and Lexical Gaps

A critical challenge in low-resource translation is the **Domain Shift** between the available parallel corpus (often religious or historical) and modern conversational needs. While our primary corpus is derived from New Testament verses, it serves as a high-quality anchor for the core syntax and grammatical morphology of Bidayuh Jagoy. To bridge the gap to everyday language, the RAG pipeline relies on **Context A (Local Glossary)** and **Context E (Synonym Bridges)**.

The local glossary is iteratively updated by native speakers to include contemporary terms (e.g., ”internet”, ”car”) that are absent from the religious text. Furthermore, the synonym bridging mechanism allows the system to pivot from an unknown modern term to a semantically similar anchor already present in the corpus, ensuring that the model maintains the correct Bidayuh Jagoy grammatical framework even when processing out-of-domain vocabulary.

4.4. Error Analysis and Fallback Mechanisms

Crucial to the scientific evaluation of the Bidayuh Jagoy Translator is the analysis of its failure modes. In cases where the RAG context fails to provide a specific Bidayuh Jagoy term (out-of-vocabulary), the system implements a **Graceful Degradation Strategy** via an Indonesian pivot. Specifically, the system prompt explicitly instructs Gemini 2.5 Flash to automatically translate missing terms into standard Indonesian (the regional lingua franca) instead of hallucinating Bidayuh-sounding morphology, explicitly flagging the insertion.

- **Example Failure**:
  - *Input*: “...the sound of a thousand drums.”
  - *Bidayuh Output*: “...okug (??? — ’seribu’) tambur.”
  - *Analysis*: Here, the term “thousand” was missing from both Context A and Context B. Instead of hallucinating a random Bidayuh-sounding word, the system followed the system prompt instruction to output the Indonesian equivalent flagged in the UI with a distinct placeholder: (???). The analysis was performed by the authors based on the log outputs.

This explicit signaling of uncertainty is a key strength. It prevents the propagation of “silent errors” and directly informs the community-driven correction loop. Native speakers can quickly identify these flags in the translation logs and provide the correct Bidayuh term (e.g., “ribu”), which is then used to update the central glossary, closing the RAG knowledge gap for future users.

5. Conclusion

The Bidayuh Jagoy Translator represents a synthesis of advanced neural search and traditional rule-based linguistics. By leveraging Transformers.js, the system democratizes access to advanced RAG tools for endangered language preservation, decoupling it from expensive server-side GPU clusters.

Importantly, while demonstrated through Bidayuh Jagoy, this hybrid client-centric RAG architecture is **applicable to any low-resource language scenario** — including regional dialects (e.g., Venetian, Sicilian), creoles (e.g., Tok Pisin), minority languages (e.g., Sorbian, Romansh), and any language on the UNESCO Red List [UNESCO 2010].

The minimal requirements are: (1) a small parallel corpus (∼2,000+ sentence pairs), (2) community involvement for validation, and (3) willingness to operate within a browser-based deployment model.

Future work will explore a "Fully Offline Mode" by integrating small-scale LLMs (e.g., Gemma 2B [Hugging Face 2025] or TinyLlama [Zhang et al. 2024]) directly into the browser via WebGPU. This would eliminate the dependency on cloud connectivity entirely, creating a truly resilient and private digital archive for the Bidayuh community. Furthermore, we aim to implement a **Graph-based RAG** approach to better capture the complex kinship and theological relations inherent in the Bidayuh Jagoy oral tradition, as well as exploring **audio-to-text integration** to support older generations of speakers who predominantly use the spoken form. To ensure continuous and consistent validation, we are developing a **WhatsApp-based validation system**, which distributes daily translation tasks (5 words or phrases per day) to voluntary native speakers, allowing for real-time iterative corpus refinement.

References

[Brown et al. 1993] Brown, P. F., Della Pietra, S. A., Della Pietra, V. J., and Mercer, R. L. (1993). The mathematics of statistical machine translation: Parameter estimation. *Computational linguistics*, 19(2):263–311. Accessed: 2025-12-22. https://aclanthology.org/J93-2003/.

[Campbell et al. 2019] Campbell, Y. M. et al. (2019). Intensifiers in Bidayuh Bau-Jagoi. *Issues in Language Studies*. Accessed: 2025-12-22. https://publisher.unimas.my/ojs/index.php/ILS/article/view/2292.

[Google AI for Developers 2025a] Google AI for Developers (2025a). *Gemini API Changelog*. Accessed: 2025-12-22. https://ai.google.dev/gemini-api/docs/changelog.

[Google AI for Developers 2025b] Google AI for Developers (2025b). Gemini API models. Accessed: 2026-04-23. https://ai.google.dev/gemini-api/docs/models/gemini.

[Google Cloud 2025] Google Cloud (2025). Gemini 2.5 Flash on Vertex AI. Accessed: 2025-12-22. https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini.

[Hugging Face 2024] Hugging Face (2024). Transformers.js. Accessed: 2025-12-22. https://huggingface.co/docs/transformers.js/index.

[Hugging Face 2025] Hugging Face (2025). Gemma 4. Accessed: 2026-04-23. https://huggingface.co/blog/gemma4.

[Maciá-Lillo et al. 2025] Maciá-Lillo, A., Mora, H., Jimeno-Morenilla, A., et al. (2025). Ai edge cloud service provisioning for knowledge management smart applications. *Scientific Reports*, 15:32246. Received: 23 December 2024; Accepted: 31 July 2025; Published: 01 September 2025; Accessed: 2026-04-23. https://www.nature.com/articles/s41598-025-14429-7.

[Malkov and Yashunin 2018] Malkov, Y. A. and Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world

graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836. Accessed: 2026-04-25. https://ieeexplore.ieee.org/document/8594636.

[Nekoto et al. 2020] Nekoto, W. et al. (2020). Participatory research for low-resource machine translation: A case study in African Languages. In *Findings of EMNLP 2020*. Accessed: 2025-12-22. https://aclanthology.org/2020.findings-emnlp.195/.

[NLLB Team et al. 2022] NLLB Team et al. (2022). No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*. Accessed: 2025-12-22. https://arxiv.org/abs/2207.04672.

[Pradhan 2025] Pradhan, R. (2025). RAG vs. fine-tuning vs. prompt engineering: A comparative analysis for optimizing ai models. *International Journal of Computer Technology, Engineering and Communication*. Accessed: 2025-12-22. https://www.researchgate.net/publication/396236634_RAG_vs_Fine-Tuning_vs_Prompt_Engineering_A_Comparative_Analysis_for_Optimizing_AI_Models.

[Supabase 2023] Supabase (2023). pgvector: Faster semantic search with HNSW indexes. Accessed: 2025-12-22. https://supabase.com/blog/increase-performance-pgvector-hnsw.

[UNESCO 2010] UNESCO (2010). Atlas of the world's languages in danger. Accessed: 2026-04-17. https://unesdoc.unesco.org/ark:/48223/pf0000187026.

[UNIMAS 2025] UNIMAS (2025). UNIMAS Institutional Repository: Philology. linguistics. Accessed: 2025-12-22. https://ir.unimas.my/view/subjects/P1.html.

[Vaswani et al. 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008. Accessed: 2025-12-22. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243-7b1b-4501-9a50-305c72d3acce-Abstract.html.

[Vercel Inc. 2025] Vercel Inc. (2025). Vercel: The platform for frontend developers. Accessed: 2025-12-22. https://vercel.com.

[Wei et al. 2022] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Accessed: 2025-12-22. https://arxiv.org/abs/2201.11903.

[Zhang et al. 2024] Zhang, P., Zeng, G., Wang, T., and Lu, W. (2024). Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*. Accessed: 2025-12-22. https://arxiv.org/abs/2401.02385.

[Zoph et al. 2016] Zoph, B., Yuret, D., May, J., and Knight, K. (2016). Transfer learning for low-resource neural machine translation. In *Proceedings of the 2016 Conference*

on Empirical Methods in Natural Language Processing, pages 1568–1575. Accessed: 2025-12-22. https://aclanthology.org/D16-1163/.

Reproducibility and Availability

The complete system is publicly available to support reproducibility and enable adaptation to other low-resource languages:

- **Production System**: https://bidayuhjagoy.vercel.app/
- **Source Code**: https://github.com/TarsilaSamille/bidayuhjagoy
- **Technology Stack**: FastAPI (backend), Vanilla JavaScript (frontend), Transformers.js (client-side AI), Supabase (pgvector database)
- **Dataset**: Available upon request to the Bidayuh community for research purposes

A. Key System Specifications

Table 1 summarises the key components, technologies, and their roles within the Bidayuh Jagoy Translator system, while Table 2 provides benchmark metrics for the client-side execution modes.

Table 1. Key System Specifications of the Bidayuh Jagoy Translator.

Component	Technology	Specification	Purpose
AI Orchestrator	Gemini 2.5 Flash	1M Token Context	RAG Synthesis
Prompt Strategy	Chain-of-Thought	5-Tiered Context	Linguistic Grounding
Local Inference	all-MiniLM-L6-v2	384-dim, WebGPU	Client-side RAG
Vector Store	Supabase/pgvector	HNSW, Cosine	Global Retrieval
Linguistic Reference	Bidayuh Jagoy	SVO Grammar	Target Language

Table 2. "Fat Client" Performance Metrics

Execution Mode	Initial Load	Inference (ms)	Memory Impact
WebGPU (Primary)	∼23MB (cached)	15–45 ms	Low (GPU-bound)
WASM (Fallback)	∼23MB (cached)	120–350 ms	Moderate (CPU)
Remote-only RAG	N/A	800+ ms (RTT)	Server-dependent